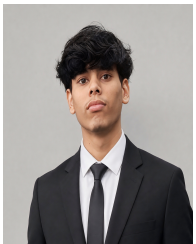


PROJECT AND TEAM INFORMATION

Project Title

RESQ — Intelligent Disaster Rescue & Emergency Response Simulator. **“Algorithms That Save Lives.”**

Student / Team Information

Team Name / Team #	RESQ Force — Team ID: DSCPP-III-2026-T117	Photo
Team member 1 (Team Lead)	Vanya Rathi – 2510383410 2510383410@geu.ac.in University Roll No.: 2028514 Phone No.: 9259011570 Section: DS-2	
Team member 2	Akshat Dobhal – 2510370353 2510370353@geu.ac.in University Roll No.: 2028631 Phone No.: 9389135760 Section: ML-2	
Team member 3	Ayush Kandwal – 2510380235 2510380235@geu.ac.in University Roll No.: 2028405 Phone No.: 9068120181 Section: ML-4	
Team member 4	Nitin Singh – 2510370229 2510370229@geu.ac.in University Roll No.: 2028786 Phone No.: 6395245688 Section: ML-1	

Project Mentor: Mr. Prabhdeep Singh

Team ID: DSCPP-III-2026-T117

PROPOSAL DESCRIPTION

Motivation

When an earthquake, a flood, a fire or a highway pile-up happens, the calls come in much faster than the number of teams, ambulances and hospital beds that are actually free. In those first few hours the operator has to decide, on a phone line, who gets help first. There is no single screen showing how badly each victim is hurt, which team is idle, or which road is still open. Two teams end up at the same spot, a serious case waits behind a minor one, and an ambulance drives into a street that collapsed an hour ago.

If we look at it as programmers, this is really a computing problem. Sorting requests by urgency, giving out limited resources and finding the fastest path on a broken road network are exactly the jobs that priority queues, graphs, Dijkstra's algorithm, BFS, stacks and linked lists are meant for. That is where the idea for RESQ came from. We wanted something small enough to run offline on any college lab machine, without internet or paid GIS software, that a control room or a mock drill could use to test dispatch decisions and to show, with numbers, that priority-based dispatch beats plain first-come-first-served.

State of the Art / Current Solution

In India most emergency help today goes through the 112 and 108 call centres. An operator notes down the call and sends the nearest free ambulance. The order is mostly the order in which calls arrive, and how urgent a case is depends on the operator's own judgement. Bigger agencies buy Computer-Aided Dispatch software such as Hexagon or Motorola PremierOne, and use ArcGIS or Google Maps for routing. On the ground, triage is done on paper using the START or SALT colour tags.

Each of these has a weak point. The commercial CAD suites are costly, closed and need a working network, so they are of little use exactly when towers and links go down. Map APIs assume the roads are fine, so they cannot know about a collapsed bridge or a flooded lane. Paper triage tags help the person standing there, but they never become a list a computer can re-order across all pending victims. Most college and research prototypes we read about solve either triage or routing, not both together. RESQ tries to put the two in one free, offline C/C++ program that also keeps a record of every decision it made.

Project Goals and Milestones

General goals. We want to build a menu-driven C/C++ simulator that keeps the disaster area as a graph of locations, takes in rescue requests, sorts victims by how serious their condition is using a priority queue, hands out rescue teams, ambulances and hospital beds, finds the shortest route with Dijkstra's algorithm, stores every operation on a stack so it can be undone or replayed, and finally prints statistics such as average response time, number of victims rescued and how busy each resource was. The last step is to run the same scenario with plain first-come-first-served dispatch and compare the two.

Milestones.

- **M1 (Week 1–2):** Requirement study, class diagram, module split; C structures for Victim and Location; linked-list victim registry.
- **M2 (Week 3):** Queue for normal requests and priority queue (min-heap on severity, tie-broken by arrival time) for critical victims.
- **M3 (Week 4):** Graph module — adjacency list of the disaster map, BFS area exploration, Dijkstra shortest-route engine.
- **M4 (Week 5):** C++ OOP layer — Victim, RescueTeam, Ambulance, Hospital, Disaster, Location, RescueOperation classes with inheritance and polymorphism.
- **M5 (Week 6):** Allocation logic, stack-based operation history with undo, sorting and searching of victims and resources.
- **M6 (Week 7):** File persistence, statistics and comparison report, console dashboard.
- **M7 (Week 8):** Testing with three scenarios (earthquake, flood, fire), documentation and final demonstration.

Project Approach and Technologies

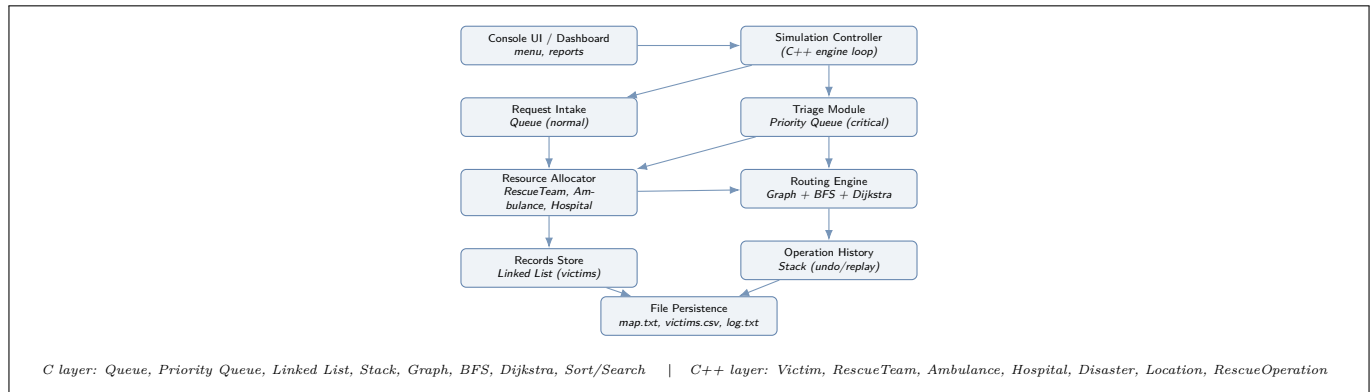
RESQ is written in two layers. The **C layer** holds the plain data structures and algorithms, and the **C++ layer** sits on top of it and turns them into an object-oriented simulation.

C — Data Structures & Algorithms. Victim and rescue records are kept in linked lists, so records can be added or removed while the simulation is running. Ordinary requests wait in a normal FIFO queue. Critical victims go into a priority queue built as a binary min-heap, ordered by a severity score from 1 (critical) to 5 (minor), with arrival time breaking ties so nobody is starved. Every completed operation is pushed on a stack, which gives us replay and undo almost for free. The disaster area itself is a weighted graph stored as an adjacency list. Nodes are places like shelters, hospitals and collapsed buildings, and each edge weight is the travel time on that road; a blocked road simply gets an infinite weight. BFS tells us which zones are still reachable, and Dijkstra gives the shortest path from the team's base to the victim and then on to the nearest hospital. Merge sort and quick sort are used to rank victims and resources, and binary or linear search pulls a record out by its ID.

C++ — OOP design. An abstract class Person is inherited by Victim and RescueTeam. Ambulance and RescueTeam both come from a common Resource base that declares a virtual `dispatch()` method, so the engine can call the same function on any unit and the right version runs. Hospital, Location, Disaster and RescueOperation keep their data private and expose it through member functions, and constructors and destructors take care of allocating and freeing memory.

Platform & tools. C11 and C++17 built with GCC/G++ on MinGW and Linux, VS Code and Code::Blocks as editors, Git and GitHub for version control, and a Makefile so each module builds separately. Maps, victims and logs are stored as plain `.txt` and `.csv` files, diagrams are drawn in draw.io, and the output is shown on a coloured console dashboard. For testing we write our own disaster scenarios and also check boundary cases such as an empty queue or a fully cut-off sector.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

- A working menu-driven RESQ simulator in C and C++, compiled with GCC, that registers victims, sorts them by severity, gives out rescue teams, ambulances and hospital beds, and prints the shortest rescue route.
- Source code kept in separate modules so each one can be reused: linked list, queue, priority queue (min-heap), stack, graph with BFS and Dijkstra, sorting and searching, plus the C++ classes.
- Three ready-to-run scenarios (earthquake, flood and fire) with sample map and victim files, so the demo does not need fresh typing every time.
- A statistics module and a short comparison report that puts priority-based dispatch next to first-come-first-served dispatch on average response time, victims rescued and resource use.
- Log files of the rescue history and the operation stack, so the run can be checked afterwards.
- Documentation: class diagram, architecture diagram, a table of time complexities, a user manual and a README on GitHub.
- A final presentation with a live run of one complete rescue cycle.

Assumptions

We assume the disaster map is given as an input file and does not change on its own; roads only open or close when the simulation triggers an event. Each victim already carries a severity number from 1 to 5, the way a first responder would tag them on the spot. Travel time is taken as proportional to the edge weight, so traffic and weather are left out. The number of rescue teams, ambulances and hospital beds is fixed when the run starts, and one ambulance handles one victim at a time. All data is either typed in or read from files, since we have no live GPS or sensor feed. Finally, the program is meant for a single user on a single machine.

References

1. Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, 4th ed., MIT Press — priority queues, BFS, Dijkstra's algorithm.
2. Bjarne Stroustrup, *The C++ Programming Language*, 4th ed. — OOP, inheritance and polymorphism.
3. Kernighan & Ritchie, *The C Programming Language*, 2nd ed. — pointers, structures, dynamic memory.
4. National Disaster Management Authority (NDMA), India — *Guidelines on Medical Preparedness and Mass Casualty Management*. <https://ndma.gov.in>
5. START / SALT mass-casualty triage protocols, U.S. Dept. of Health & Human Services. <https://chemm.hhs.gov/startadult.htm>
6. GeeksforGeeks — Graph representations, Dijkstra and heap implementations. <https://www.geeksforgeeks.org>
7. E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, 1959.
8. Emergency Response Support System (ERSS-112), Ministry of Home Affairs, India. <https://112.gov.in>